63. Операторы управления потоком выполнения программ

При запуске программы, центральный процессор (сокр. *«ЦП»*) начинает выполнение кода с первой строки функции main(), выполняя определенное количество стейтментов, а затем завершает выполнение при завершении блока main(). Последовательность стейтментов, которые выполняет ЦП, называется **порядком выполнения программы** (или ***«потоком выполнения программы»***).

Оглавление:

1. Порядок выполнения программ
2. Остановка
3. Прыжок
4. Условные ветвления
5. Циклы
6. Исключения
7. Заключение

Порядок выполнения программ

Большинство программ, которые мы рассматривали до этого момента, были **линейными** с **последовательным выполнением**, то есть порядок выполнения у них один и тот же каждый раз: выполняются одни и те же стейтменты, даже если значения, которые вводит пользователь, — меняются.

Но на практике это не всегда может быть то, что нам нужно. Например, если мы попросим пользователя сделать выбор между `+` и `/`, а пользователь введет некорректный символ (например, `*`), то в идеале нам нужно было бы попросить его ввести символ еще раз. Но это невозможно в линейной программе. Кроме того, бывают случаи, когда нужно выполнить что-то несколько раз, но количество этих повторений наперед неизвестно. Например, если бы мы хотели вывести все целые числа от `0` до числа, которое введет пользователь, то в линейной программе мы бы не смогли это сделать, не зная наперед число, которое введет пользователь.

К счастью, в языке C++ есть **операторы управления порядком выполнения программы**, которые позволяют программисту изменить поток выполнения программы центральным процессором.

Остановка

Самый простой оператор управления порядком выполнения программы — это **остановка**, которая сообщает программе немедленно прекратить свое выполнение. В языке C++ остановка осуществляется с помощью **функции exit()**, которая определена в **заголовочном файле** cstdlib. Функция exit() принимает целочисленный параметр, который затем возвращает обратно в операционную систему в качестве кода выхода. Например:

```
#include <iostream>
#include <cstdlib> // для функции exit()

int main()
{
    std::cout << 5;
    exit(0); // завершаем выполнение программы и возвращаем 0 обратно в операционную систему

    // Следующие стейтменты никогда не выполнятся
    std::cout << 3;
    return 0;
}
```

Прыжок

Следующим оператором управления порядком выполнения программы является **прыжок** (или ***«переход»***). Он безоговорочно сообщает компилятору во время выполнения перейти от одного стейтмента к другому, т.е. выполнить прыжок. **Ключевые слова goto**, **break** и **continue** являются разными типами прыжков, об их различиях мы поговорим несколько позже.

Вызовы функций — это тоже, в некоторой степени, прыжки. При выполнении вызова функции, ЦП переходит к началу вызываемой функции. Когда вызываемая функция заканчивается, выполнение возвращается к следующему стейтменту, который находится после вызова этой функции.

Условные ветвления

Условное ветвление заставляет программу изменить свой порядок выполнения, основываясь на значении результата выражения. Одним из основных операторов условного ветвления является if, который вы уже могли видеть в программах раньше. Например:

```
int main()
{
    // Делаем A
    if (expression)
        // Делаем B
    else
        // Делаем C

    // Делаем D
}
```

Здесь есть два возможных пути выполнения программы. Если результатом выражения будет true, то программа выполнит `A`, `B` и `D`. Если же результатом выражения будет false, то программа выполнит `A`, `C` и `D`. Таким образом, выполнится либо `B`, либо `C`, оба варианта выполняться вместе не будут. Это уже не линейная программа.

Ключевое слово switch также предоставляет механизм для выполнения условного ветвления. Более подробно об операторах if и switch мы поговорим на соответствующих уроках.

Циклы

Цикл заставляет программу многократно выполнять определенное количество стейтментов до тех пор, пока заданное условие не станет ложным. Например:

```
int main()
{
    // Делаем A
    // B делается в цикле 0 или больше раз
    // Делаем C
}
```

Эта программа может выполняться как `ABC`, `ABBC`, `ABBBC`, `ABBBBC` или даже `AC`. Опять же, она больше не является линейной, её порядок выполнения зависит от того, сколько раз выполнится цикл (если вообще выполнится).

В языке C++ есть 4 типа циклов:

- цикл while;
- цикл do while;
- цикл for;
- цикл foreach (добавили в C++11).

Мы подробно рассмотрим каждый из них в этой главе, кроме foreach (о нем немного позже).

Исключения

Исключения предлагают механизм обработки ошибок, возникающих в функции. Если в функции возникает ошибка, с которой она не может справиться, то она может выбросить исключение. Это заставит ЦП перейти к ближайшему блоку кода, который обрабатывает исключения данного типа.

Обработка исключений — это довольно сложная тема и это единственный тип оператора управления порядком выполнения, который мы не будем рассматривать в этой главе.

Заключение

Используя операторы управления порядком выполнения программы, вы можете повлиять на поток выполнения программы центральным процессором, а также на то, из-за чего он может быть прерван. До этого момента функционал наших программ был очень ограничен. Теперь же с операторами управления порядком выполнения программ мы сможем осуществить огромное количество разных интересных вещей, например, отображение меню до тех пор, пока пользователь не сделает правильный выбор; вывод каждого числа между `x` и `y`, и много-много чего еще.

Как только вы разберетесь с этой темой, вы перейдете на новый, более качественный уровень. Больше вы не будете ограничены игрушечными программами или простенькими упражнениями — вы сможете писать полноценные программы. Вот именно здесь и начинается всё самое интересное.